SDG BLOCKCHAIN ACCELERATOR

Prototype (PoC) Report

1. Project Information

- **Project Name:** SHELTER
- **Challenge & UNDP Office:** MALAWI UNDP
- **Document Version:** 1.0

2. Project Overview

SHELTER is a decentralized humanitarian aid distribution system built on the Cardano Blockchain. It addresses the challenge of distributing funds to unbanked populations in Malawi by utilizing a "Low Tech Wallet" infrastructure. The prototype allows an ADMIN/NGO to lock funds into an Aiken smart contract. These funds are time-locked and destined for a specific Beneficiary, who can claim them using a basic feature phone via SMS. The transaction is facilitated by a local Merchant, who utilizes a dedicated Web Platform and Dashboard to create orders, validate OTPs, and settle payments on-chain. The system ensures transparency, prevents fraud through cryptographic verification, and allows the recovery of unused funds.

3. Repository Structure

```
/
    /contracts   → Aiken smart contract source code & Tests
    /ltw_service       → LTW Service & Tests & README.md
    /merchant_platform     → Merchant platform & Tests & README.md
    /docs        → README.md, deployment guide, screenshots
```

4. Build Instructions

1. Smart Contracts:

```
cd contracts
aiken build
# Output: Generated plutus.json in /contracts/plutus.json
```

2. LTW Service:

```
cd ltw_service
npm run build
```

3. Merchant Platform:

```
cd merchant_platform
npm run build
```

5. Test Instructions

1. Smart Contracts Tests:

```
cd contracts
aiken test
```

2. LTW Service Tests:

```
cd ltw_service
npm run test
```

3. Merchant Platform Tests:

```
cd merchant_platform
npm run test
```

6. Deployment Instructions

To deploy the SHELTER prototype to the Cardano Preview Testnet, follow these steps:

1. **Build Smart Contract:** Ensure you run aiken build inside the /contracts folder. This will generate a plutus.json file as output, which is required by the LTW Service.
2. **Environment Setup:**
   - **2.1.** Configure your ltw_service/.env file following the structure defined in ltw_service/.env.example.
     - **2.1.1.** Important: Pre-generate the xprv (Extended Private Key) for the STEWARD_KEY and ensure it has sufficient funds.

2.1.2. It is recommended to have multiple UTxOs available in this wallet to handle concurrent beneficiary assignments and avoid transaction contention errors.

2.1.3. Note: The value for COMPILE_CODE_PLUTUS should be extracted from the compiledCode key within the generated plutus.json file.

2.2. Configure your merchant_platform/.env file following the structure defined in merchant_platform/.env.example.

3. **Start LTW Service**

```
cd ltw_service
make devup
```

4. **Start Merchant Platform**

```
cd merchant_platform
npm run dev
```

5. **Usage**

All endpoints require a valid JWT for authentication. Below are the steps to generate and use JWTs:

- **Create User**: To create a new user, send a POST request to http://localhost:3000/api/users with the following JSON body:

```
{
  "email": "<EMAIL>",
  "password": "<PASSWORD>"
}
```

Replace <EMAIL> with the user's email and <PASSWORD> with the desired password.

- **Login**: To generate a JWT, send a POST request to http://localhost:3000/api/users/login with the following JSON body:

```
{
  "email": "<EMAIL>",
  "password": "<PASSWORD>"
}
```

Replace <EMAIL> with the user's email and <PASSWORD> with their password. The response will include the JWT.

Once you have the JWT, include it in the Authorization header of all subsequent API requests:

Authorization: Bearer <JWT>

These commands enable secure user authentication and authorization. Below are additional operations available via the API:

- **Add Beneficiary**: To add a new beneficiary to the system, send a POST request to http://localhost:3000/api/beneficiaries with the following JSON body:

```
{
  "phoneNumber": "<PHONE_NUMBER>"
}
```

Replace <PHONE_NUMBER> with the beneficiary's phone number.

- **Create Order**: Merchants can create orders to charge a beneficiary as follows:

Send a POST request to http://localhost:3000/api/beneficiaries/order with the following JSON body:

```
{
```

```
  "amount": <AMOUNT>,

  "token": "<TOKEN>",

  "phoneNumber": "<PHONE NUMBER>",

  "merchAddress": "<MERCH_ADDRESS>",

  "description": "<DESCRIPTION>"

}
```

Replace <AMOUNT> with the order amount, "<TOKEN>" with the token identifier, "<PHONE NUMBER>" with the beneficiary's phone number, "<MERCH_ADDRESS>" with the merchant's blockchain address, and "<DESCRIPTION>" with additional information or context about the order (optional).

- **Complete Order**: Merchants can complete an order by validating the OTP received by the beneficiary. Send a POST request to http://localhost:3000/api/beneficiaries/complete-order with the following JSON body:

```
{

  "phoneNumber": "<PHONE NUMBER>",

  "otp": "<OTP>"

}
```

Replace "<PHONE NUMBER>" with the beneficiary's phone number and "<OTP>" with the OTP received.

- **Deactivate Account**: To deactivate a beneficiary's account, send a POST request to http://localhost:3000/api/beneficiaries/deactivate-account with the following JSON body:

```
{
```

```
"phoneNumber": "<PHONE_NUMBER>"

}
```

Replace <PHONE_NUMBER> with the phone number of the account you wish to deactivate.

- Get Token Balance: To retrieve the balance for a beneficiary, send a GET request to http://localhost:3000/api/beneficiaries/<PHONE_NUMBER>/balance.

Example:

```
GET http://localhost:3000/api/beneficiaries/+1234567890/balance
```

Replace <PHONE_NUMBER> with the beneficiary's phone number.

- **Latest Orders:** Retrieve the most recent orders for a specific beneficiary.

```
GET http://localhost:3000/api/orders/beneficiaries/<PHONE_NUMBER>?limit=<LIMIT>
```

Replace <PHONE_NUMBER> with the beneficiary's phone number. The request body should be empty.

- **Order Status**: Check the status of a specific order.

```
GET http://localhost:3000/api/orders/<ORDER_ID>/status
```

Replace <ORDER_ID> with the unique ID of the order. The request body should be empty.

- **Status**: Check the health status of the server.

```
GET http://localhost:3000/status
```

No authentication token is required. The request body should be empty.

- **Metrics**: Retrieve system metrics for monitoring.

```
GET http://localhost:3000/metrics
```

No authentication token is required. The request body should be empty.

Once activated, the beneficiary has access to the following SMS commands:

- **Check Balance**: The beneficiary can check their available balance using the command:

```
BALANCE
```

- **Get Wallet Address**: The beneficiary can retrieve the wallet address associated with their phone number using the command:

```
ACCOUNT
```

- **Update Phone Number**: The beneficiary can update their registered phone number using the command:

```
UPDATE_PHONE <NEW_PHONE_NUMBER>
```

- **Confirm Phone Number**: After receiving the OTP on the new phone number, confirm it using the command:

```
CONFIRM_PHONE <OTP>
```

The merchant can also perform the following actions via API:

- View stats:

```
GET http://localhost:3000/api/orders/merchant/stats
```

- View Orders (General or Specific Beneficiary)

```
GET http://localhost:3000/api/orders/merchant/<BENEFICIARY_PHONE_NUMBER>
```

7. Testnet / Emulator Results

Transaction Type	Status	CPU	Memory	Notes
Assign AID	Success	N/A	N/A	✅ **Passed OK:** Standard transaction confirmed by Ledger. TxHash:f5cea9526cec021b74fc35fdec275ea24ddfb7221026f49c493c10d32da8298c
Consume AID	Success	3B	7M	✅ **Passed OK:** Validator execution within limits. TxHash:be1b89805134ad076f2a0ddb45c681d900138679b30629dcdaaf008626b6fbbc

Recover AID	Success	3B	7M	✅ **Passed OK:** Validator execution within limits. TxHash:051680893a2e67106a5cd5cd7be3cbea58e05b31c27d677798c98c486b17bc9f
Consume AID (without sign beneficiary)	Reject	.	.	✅ **Test Passed:** Security blocked action. Error: *BeneficiaryAction, must be signed with beneficiary key*
Consume AID (before claim start)	Reject	.	.	✅ **Test Passed: Time-lock active. Error: *BeneficiaryAction, must be valid after claim_start***
Consume AID (after claim end)	Reject	.	.	✅ **Test Passed: Expired AID blocked. Error: *BeneficiaryAction, must be valid before claim_end***
Recover AID (without sign steward)	Reject	.	.	✅ **Test Passed:** Security blocked action. Error: *StewardAction, must be signed with steward key*
Recover AID (before claim end)	Reject	.	.	✅ **Test Passed: Active AID protected. Error: *StewardAction, must be valid after claim_end***

8. Dependencies & Environment

Blockchain & Smart Contracts

- Cardano Preview Testnet
- Aiken v1.1.17
- Vodka / Cocktail / Mocktail
- Cardano Node & CLI
- Blockfrost

Frontend Ecosystem (Merchant Platform)

- **Framework:** Next.js 16 (React 19)
- **UI & Styling:** Tailwind CSS v4 & Radix UI
- **Blockchain Integration:** Mesh SDK (@meshsdk/core)
- **Validation:** Zod & React Hook Form
- **Testing:** Vitest & Playwright

Backend Ecosystem (LTW Service)

- **Runtime:** Node.js (v20+) & TypeScript
- **Testing:** Jest
- **Blockchain Integration:** Mesh SDK (@meshsdk/core)
- **Integrations:** Twilio SDK & Axios
- **Database:** PostgreSQL & FerretDB
- **Infrastructure:** Docker

ENV_TEMPLATE_LTW_SERVICE

```
# APP
JWT_SECRET=
JWT_EXPIRATION_TIME=
APP_PORT=
PRODUCTION=
TTL_AS_MINUTES=
ENABLE_BACKDOOR_CMD=
SMS_NOTIFICATION_ON_CREATE_BENEFICIARY=
SMS_ACTIVE_CHANNEL=
LOG_DEBUG=
LOCAL_KEY=

# MULTILANGUAGE
LANGUAGE=
PRELOAD_LANGUAGES=

# TWILIO
TWILIO_ACCOUNT_SID=
TWILIO_AUTH_TOKEN=
TWILIO_WEBHOOK_URL=
MESSAGING_SERVICE_SID=

# KOTANI
KOTANIPAY_API_KEY=
KOTANIPAY_API_URL=

# DATABASE
```

```
DB_USER=
DB_PASS=
DB_NAME=
DB_PORT=

MONGO_EXPRESS_USER=
MONGO_EXPRESS_PASS=

# CARDANO
BLOCKFROST_KEY=
STEWARD_KEY=
COMPILE_CODE_PLUTUS=
SLOT_TIME_SECONDS=
SAFETY_MARGIN_SECONDS=
BLOCKS_TO_WAIT_FOR_CONFIRMATION=
```

`SMS_ACTIVE_CHANNEL`

Specifies which SMS provider to use for sending messages. Available options are:

- `twilio` - Use Twilio as the SMS provider.
- `kotanipay` - Use KotaniPay as the SMS provider.

`SMS_NOTIFICATION_ON_CREATE_BENEFICIARY`

Controls whether an SMS notification is sent automatically when a new beneficiary is created via the API.

- `1` - **Enable** SMS notifications on beneficiary creation.
- `0` - **Disable** SMS notifications on beneficiary creation.

ENV_TEMPLATE_MERCHANT_PLATFORM

```
NEXT_PUBLIC_API_BASE_URL=
NEXT_PUBLIC_API_TIMEOUT=
NEXT_PUBLIC_BLOCKFROST_KEY=
```

9. Demo / Walkthrough

https://drive.google.com/file/d/1SQWcmXssbAkONgnC3icQdrEutoaTCxs2/view?usp=sharing

The video demonstrates the complete end-to-end lifecycle of the SHELTER system:

1. **Beneficiary Registration:** The beneficiary is registered in the system.
2. **Wallet Initialization:** The Beneficiary receives a welcome SMS containing their unique wallet address.
3. **AID Assignment:** Funds are assigned and locked for the beneficiary.
4. **Balance Inquiry:** The Beneficiary sends an SMS command to check their balance and receives an immediate response with the available amount.
5. **Order Creation:** The Merchant initiates a new order using the **Merchant Web Platform**.
6. **OTP Verification:** The Beneficiary receives an SMS with the order details and a One-Time Password (OTP), which they dictate to the Merchant.
7. **Order Fulfillment:** The Merchant enters the OTP into the platform to complete the transaction.
8. **Settlement Confirmation:** The Beneficiary receives a final SMS confirming the payment and displaying their updated AID balance.

10. Remaining Issues / Next Steps

- **Production Refactoring (Trace Removal)**: Refactor Aiken contract debug messages (trace) into expect statements to strip string literals, reducing script size and transaction fees for Mainnet deployment.
- **LTW Service Optimization**: Refactor backend components to improve modularity and expand unit/integration test coverage (using Jest) to >80% coverage.
- **Merchant Platform Refactoring**: Implement automated component testing (using Vitest) to ensure UI reliability.
- **Preprod Stress Testing**: Migrate the testing environment from Preview to Preprod to simulate high-concurrency scenarios.
- **Security Audit**: Conduct an external security audit focusing on the time-lock logic and backend key management.
- **Rahat Integration**: Evaluate the incorporation of the Rahat Dashboard to enhance the Admin/NGO interface with industry-standard impact metrics and interoperable aid distribution tools.